

EDUCATION SERIES #2

Business Case Scenario : From Prompt to Agent

Five layers — prompt, context, harness, loop, graph — taught through one business story: Solstice Outfitters builds a support agent, week by week.

Ashley Castelino · 30-minute session

One build, five weeks, five layers

01	Prompt engineering	Instructions as an interface contract
02	Context engineering	The model only knows what you send
03	Harness engineering	The code you write around the model
04	Loop engineering	How a model becomes an agent
05	Graph engineering	Structuring many loops into a system

Solstice Outfitters is drowning in support tickets

- Outdoor-gear store, four-person support team, a queue that never empties.
- 80% of tickets are routine: order status, refunds, product questions.
- Your brief: build an agent that resolves the routine ones — safely.

Solstice Outfitters is fictional; the constraints are typical of a small e-commerce team.

Monday, 9 AM: the queue Maya's ticket is stuck in

S Solstice Outfitters
Support desk

Open tickets142

Unassigned87

Urgent23

Waiting on customer31

Solved this week64

Open tickets

Median first reply: 9h 40m · Team online: 2 of 4

OPEN	Maya Rodriguez #4521 · maya.r@gmail.com	Where is my order? Hi — I ordered the Ridgeline 2P tent two weeks ago and nothing has arrived...	2d
OPEN	Dan Okafor #4526 · dokafor@...	Refund for duplicate charge I was charged twice for order #4477, can someone reverse the second...	6h
NEW	Priya Shah #4530 · pshah@...	Sizing — Cascade rain shell Does the women's medium fit like the 2024 model or smaller? I'm between...	1h
WAITING	Tom Berger #4499 · tberger@...	Wrong color shipped Ordered the juniper green pack, received orange. Happy to swap if...	3d
OPEN	Lena Fischer #4514 · lfischer@...	Order status — #4514 Tracking hasn't updated since Tuesday, is my order stuck somewhere...	1d
NEW	Ash Patel #4531 · apatel@...	Warranty on trekking poles One of the locking levers snapped on the second hike. What does the...	24m

Ticket #4521 — Maya's missing tent — is the one we'll follow for the rest of the talk.

UI mockup; company and data are fictional.

The brief, translated from business asks into engineering

What the business needs	What that means technically
Answer “where is my order?” instantly	lookup_order tool calling the order API
Refunds follow policy; a human signs off on money	Policy in the prompt; approval guardrail in the harness
Replies match our published help docs	search_docs retrieval into the context window
We can see what the agent did, and why	Every turn of the loop logged and auditable

Each section ends in a live-code demo. Everything shown is provider-agnostic.

01

Prompt engineering

Week 1: the agent needs a job description — role, policy, examples, output format. That is the prompt.

Prompt engineering

Writing the model's instructions the way you'd write an interface contract: role, task, constraints, examples, and the exact output format you expect back.

WHY IT MATTERS

Highest leverage per unit of effort — no retraining, no new infrastructure, just better instructions. Version it like code.(Imagine you are teaching someone for the job)

Same model, two prompts, very different tickets

Vague

“You are a helpful support agent. Answer the customer.”

Refund policy gets improvised.

Output format changes run to run.

Engineered

Role + policy: refunds within 30 days, $\leq$ \$200.

Two worked example replies (few-shot).

Output schema: JSON with reply, action, confidence.

Treat the prompt as versioned code: reviewed, tested, diffed.

The engineered prompt, in full

You are the support agent for Solstice Outfitters.

POLICY

- Refunds: within 30 days of delivery, max \$200 – otherwise escalate.
- Never promise delivery dates; quote the carrier's estimate.

OUTPUT – respond with JSON only:

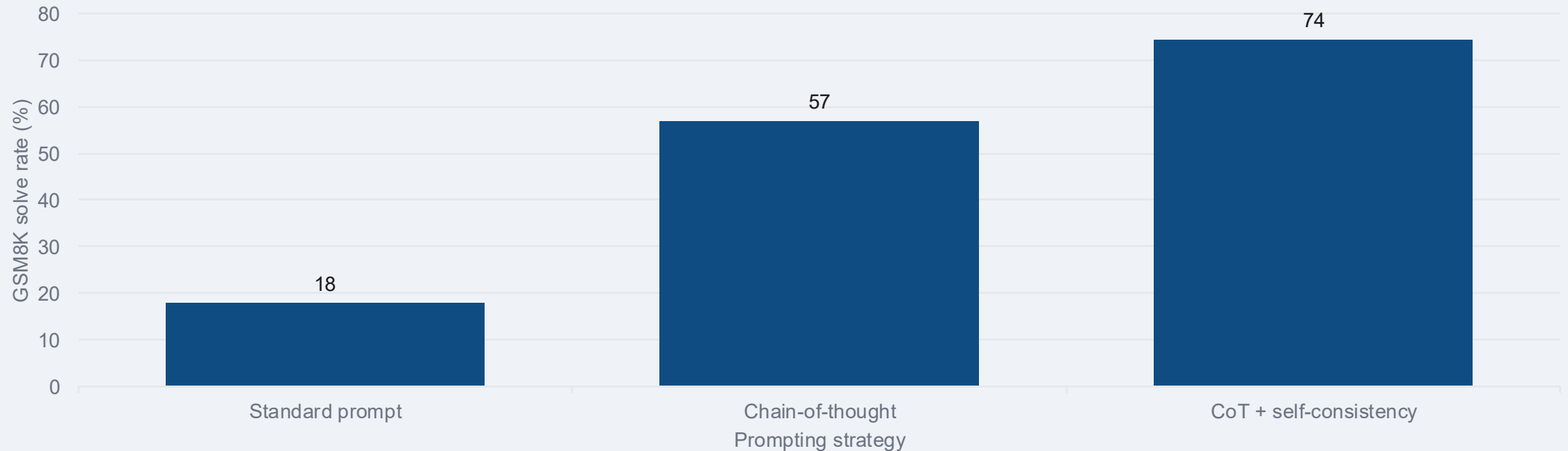
```
{ "reply":      "<message to the customer>",  
  "action":     "none | refund | escalate",  
  "confidence": 0.0 - 1.0 }
```

EXAMPLES: two ideal ticket -> reply pairs pasted here (few-shot).

Role, policy, schema, examples — the whole contract, versioned in git next to the code.

EVIDENCE

Asking the model to reason step-by-step tripled math accuracy



Same model, same questions — only the prompt changed.

Wei et al. 2022; Wang et al. 2022 — PaLM 540B on GSM8K math word problems.

Four techniques cover most of what prompt engineering does

Technique	What it does	In our support agent
Role + constraints	Pins tone and policy limits	Refund rules stated up front
Few-shot examples	Shows the target, not tells it	Two ideal replies embedded
Output schema	Makes responses parseable	JSON: reply, action, confidence
Chain-of-thought	Buys reasoning on hard cases	“Check policy before refunding”



LIVE CODE · PROMPT ENGINEERING

Demo: the prompt against real tickets

We'll run the vague and the engineered prompt on the same three tickets and diff the JSON that comes back.

Switching to the editor — the deck picks back up when we're done.

02

Context engineering

Week 2: the agent invents order details. It doesn't need discipline — it needs the right facts in the window.

Every API call starts from zero — context is all the model gets

- No memory between calls: “conversation” is you resending the history.
- The window is finite — long inputs crowd out instructions.
- Quality drops before limits do: models lose the middle of long contexts.

“Lost in the middle”: Liu et al., 2023.

The harness assembles the context fresh on every call



For ticket #4521 the agent sees the refund policy, three turns, and one order record — not the database.

Spend the window like a budget, not a dumping ground

- Retrieve the three relevant docs; don't paste the manual.
- Summarize or drop old turns — keep decisions, not chatter.
- Put critical instructions first and last; the middle gets skimmed.

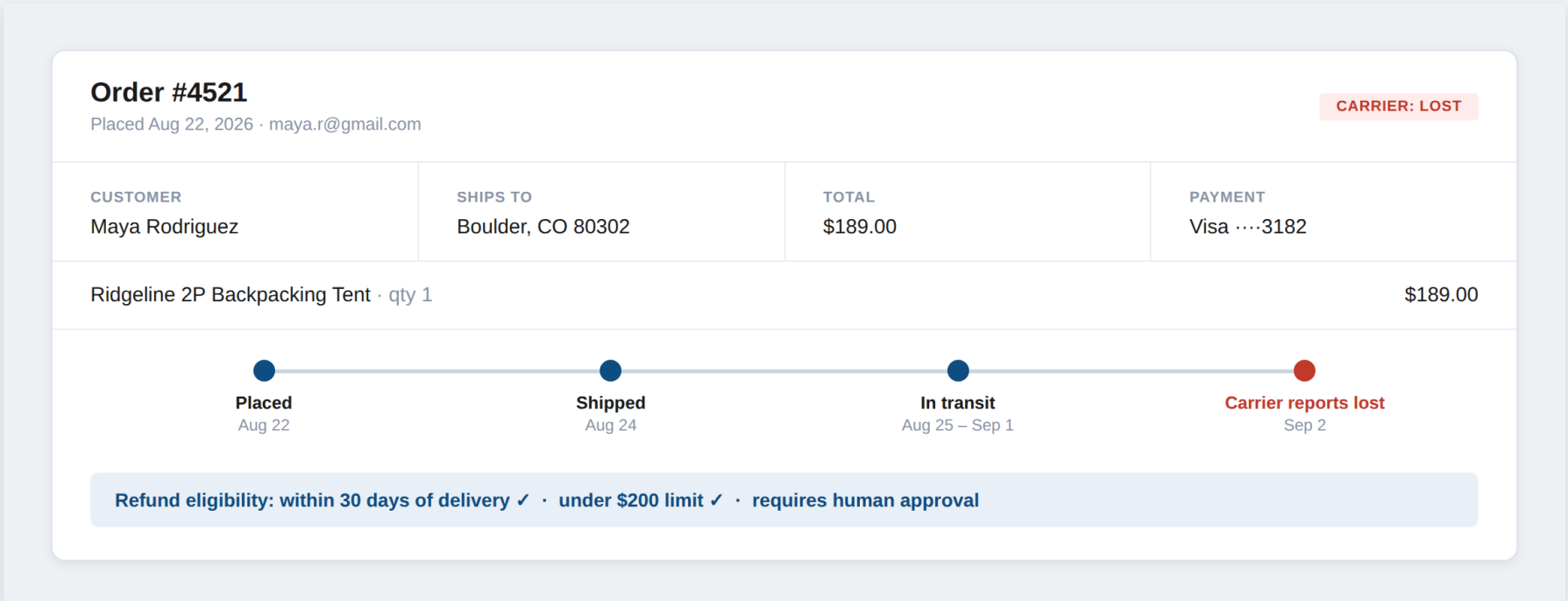
What the window actually holds for ticket #4521

```
[system]      Role, policy, tool specs           ~600 tokens
[history]     Turns 1-3, summarized to one line  ~150
[retrieved]   Order 4521 — shipped Aug 24, carrier: LOST ~120
[retrieved]   Help docs, "Lost packages" (top match) ~200
[user]        "It's been two weeks. Where is my order?" ~20
-----
Total sent to the model                               ~1,090 tokens
```

Everything the model will ever know about this ticket — chosen, not dumped.

Token counts illustrative.

“Retrieved data” is just this record, fetched at the right moment



The refund policy check is answerable only because this record is in the window.

UI mockup; data fictional.



LIVE CODE · CONTEXT ENGINEERING

Demo: assembling the window

We'll build the context for ticket #4521 piece by piece and watch what retrieval adds — and what it costs.

Switching to the editor — the deck picks back up when we're done.

03

Harness engineering

Week 3: the prompt can't call the order API. Time to write the code around the model.

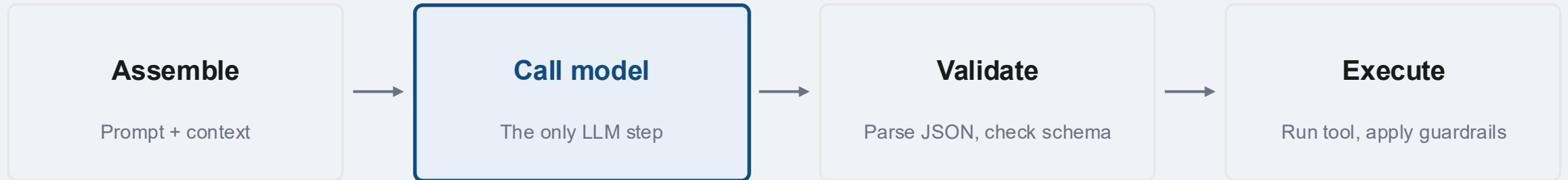
Harness

The application code **wrapped around the model**: it assembles **context**, **exposes tools**, **parses and validates output**, **retries failures**, and **enforces guardrails**.

WHY IT MATTERS

The model never runs a tool or touches your database — the harness does. Its permissions, not the model's intent, bound what can happen.

One turn of the harness, end to end



Only the shaded step is the LLM. Everything else is deterministic code — testable, loggable, yours.

The whole harness is one short function

```
def handle_turn(ticket):
    ctx = assemble(SYSTEM_PROMPT,          # layer 1: the prompt
                   summarize(ticket.history), # layer 2: context
                   retrieve(ticket))
    out = call_model(ctx, tools=TOOLS)      # the only LLM step
    act = validate(out)                    # parse JSON, check schema
    if act.tool == "refund_order":
        require_human_approval(act)        # guardrail
    return execute(act)                   # run the tool
```

Layers 1 and 2 are just arguments here. The loop — next — is what calls this repeatedly.



LIVE CODE · HARNESS ENGINEERING

Demo: `handle_turn`, end to end

One turn against a test ticket: assemble, call, validate — and what happens when the model returns bad JSON.

Switching to the editor — the deck picks back up when we're done.

04

Loop engineering

Week 4: one call handles one step, but ticket #4521 takes four. Wrap the harness in a loop.

Agentic loop

Call the model → if it requests a tool, run it → append the result to context → call again.
Repeat until the model answers instead of acting.

WHY IT MATTERS

The model decides what happens next at every step. That decision-making inside a loop is what “agent” actually means.

The loop is ten lines wrapped around the harness

```
def run_agent(ticket, max_turns=8):          # cap prevents runaways
    history = []
    for _ in range(max_turns):
        act = handle_turn(ticket, history)    # the harness, from layer 3
        if act.tool is None:                  # model chose to reply
            return act.reply                  # -> the exit condition
        result = execute(act)                 # run the requested tool
        history.append((act, result))         # context grows each turn
    return escalate(ticket)                   # budget hit -> human
```

The model steers; the code decides how long it's allowed to drive.

Ticket #4521 through the loop, turn by turn

- 1 User: “Ordered two weeks ago, nothing arrived.” Model calls `lookup_order(4521)`.
- 2 Harness runs it — shipped 12 days ago, carrier reports lost — and appends the result.
- 3 Model reads it, calls `refund_order(4521)`. Guardrail: pause for human approval.
- 4 Approved → harness refunds. Model writes the apology; no tool call → loop exits.

Four model calls, two tools, one resolved ticket.

The same four turns, as Maya and the team see them

Where is my order? Ticket #4521 · Maya Rodriguez RESOLVED · 4 MIN

Maya Rodriguez · 9:12 AM

Hi — I ordered the Ridgeline 2P tent two weeks ago and nothing has arrived. Order #4521. Can you tell me what's going on?

agent → `lookup_order(4521)` · shipped Aug 24 · carrier status: **LOST**

agent → `refund_order(4521, reason="lost")` · \$189.00 · **awaiting approval**

✓ **Approved** by J. Park (support lead) · 9:15 AM

Solstice Assistant · 9:16 AM

I'm so sorry, Maya — your Ridgeline 2P tent was lost in transit by the carrier. I've issued a full refund of **\$189.00** to your original payment method; it should appear within 3–5 business days. If you'd like to reorder, I can flag it for priority shipping.

Tool calls and the approval show up as internal events; Maya sees one apology, four minutes in.

UI mockup; conversation fictional.

Loops fail in predictable ways — cap, compact, and gate them

- Runaway: cap iterations and spend per ticket.
- Context bloat: every turn appends; compact as you go.
- Destructive tools: approvals live in the harness, not the prompt.



LIVE CODE · LOOP ENGINEERING

Demo: the loop resolves #4521

run_agent live: tool calls streaming, the approval pause, and the exit when the model chooses to reply.

Switching to the editor — the deck picks back up when we're done.

05

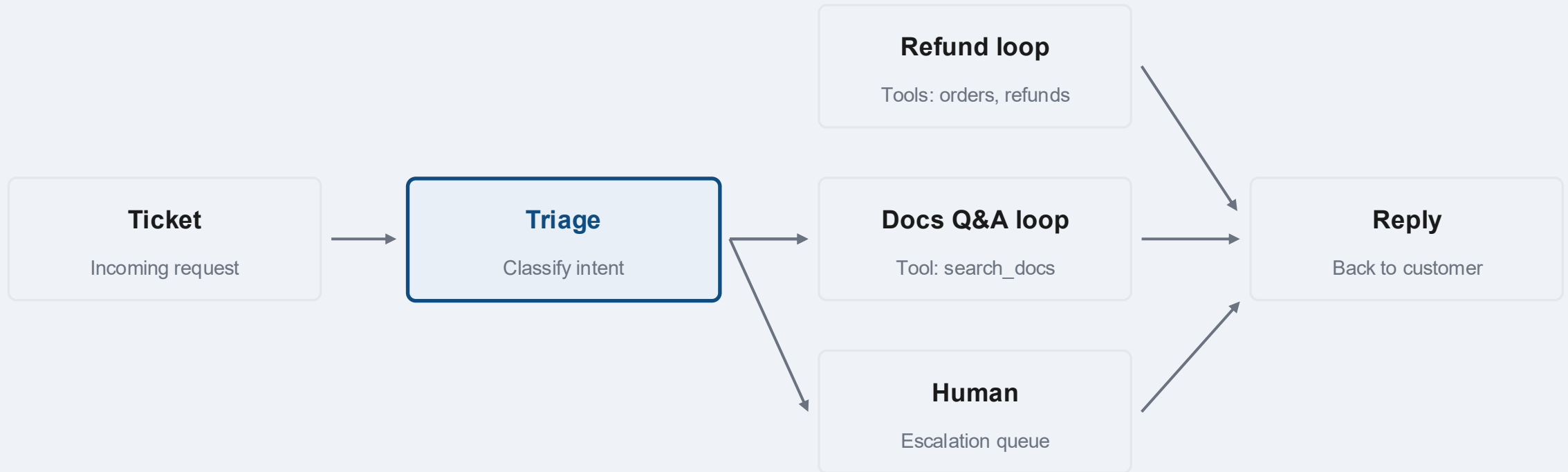
Graph engineering

Week 5: refunds, sizing questions, escalations — one loop can't do every job well. Add structure: a graph.

A graph gives loops structure: nodes do work, edges decide routing

- Specialists beat one giant prompt — each node gets a small, focused context.
- Edges encode policy in code: what routes where, what needs approval.
- Same idea behind LangGraph, and subagents in the Claude Agent SDK.

The support agent as a graph: triage routes, specialists loop



Each specialist is its own small agentic loop; escalation is just an edge, not a failure.

The graph is data: nodes registered, edges routed

```
g = Graph()
g.add_node("triage", triage_agent)      # each node: its own small loop
g.add_node("refunds", refund_agent)
g.add_node("docs", docs_agent)
g.add_node("reply", merge_and_send)
g.add_edge("triage", route_by_intent)  # edges decide where work goes
g.add_edge("refunds", "reply")
g.add_edge("docs", "reply")
app = g.compile(checkpointing=True)     # pause/resume between nodes
```

Pseudocode in the LangGraph style — the Claude Agent SDK expresses the same shape with subagents.

One messy ticket through the graph

- 1 Ticket: “Package never arrived — and your sizing chart is wrong.”
- 2 Triage finds two intents and routes one to each specialist.
- 3 Refund loop resolves order 4521; Docs Q&A fixes the sizing answer — in parallel.
- 4 Reply node merges both into one message back to the customer.

Two loops, one reply — the routing lived in edges, not in a prompt.

Single loop or graph? Let the workflow decide

Single loop

One job, one toolset, short horizon.

Simplest to build, debug, and eval.

Our refund agent alone is exactly this.

Graph

Distinct stages, routing, or parallel work.

Checkpoints and human gates between nodes.

The full support pipeline needs it.

Start with one loop; move to a graph when routing outgrows the prompt.



LIVE CODE · GRAPH ENGINEERING

Demo: triage routes a messy ticket

The compiled graph on Maya's two-intent ticket: routing, both specialists in parallel, one merged reply.

Switching to the editor — the deck picks back up when we're done.

What shipping this buys Solstice

- Routine tickets resolved in minutes, around the clock.
- The four-person team handles judgment calls, not copy-paste replies.
- Every action logged, gated, and inside policy — auditable by design.

Fictional scenario — measure your own deflection rate and CSAT before claiming numbers.

What to take away

- 01 Prompt = interface contract: role, constraints, examples, output schema.
- 02 Context is all the model knows — assemble it deliberately, spend it like a budget.
- 03 The harness is your code around the model; it holds the real permissions.
- 04 Loop: model acts, harness executes, repeat until it replies. That's an agent.
- 05 Graph: many small loops with routing on the edges — structure for real workflows.

Questions?

GO DEEPER

Wei et al., Chain-of-Thought Prompting Elicits Reasoning (2022) — the CoT numbers

Anthropic, Building Effective Agents — patterns for loops, routing, and graphs

Liu et al., Lost in the Middle (2023) — why context position matters